1 Restaurant Recommender

1.1 Overview

The **ForkCast Recommender** is a collaborative filtering engine that suggests restaurants to users based on their review history. It uses the Surprise library's Singular Value Decomposition (SVD) implementation to generate predictions from user-item rating data.

1.2 1. Review Data Format

All review data is stored in a single CSV file named `combined_reviews.csv`, with the following format (pipe-delimited):

restaurantName|reviewerId|rating|reviewDate

Each row represents one review by a specific user of a specific restaurant, along with a numeric rating (1–5) and the date of review.

- Users may review multiple restaurants
- Restaurants may receive reviews from multiple users
- Duplicate reviews are prevented at submission time

1.3 2. SVD Model Training

The core of the recommendation logic is built using Surprise's matrix factorization algorithm (SVD). This model is retrained every time a new review is submitted and accepted.

```
from surprise import Dataset, Reader, SVD

reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(df[["user", "item", "rating"]], reader)
trainset = data.build_full_trainset()

model = SVD()
model.fit(trainset)
```

After training, predictions can be made using:

```
model.predict(user_id, restaurant_name).est
```

Predicted scores are used to rank unseen restaurants and generate recommendations.

1.4 3. Logic

Users must submit at least two reviews to receive recommendations. This ensures that the system has a minimal amount of information to begin making predictions.

The recommendation pipeline follows these steps:

1. Review is submitted via the web interface
2. The system checks for duplicates in combined_reviews.csv
3. If the review is new, it is appended to the CSV
4. The recommender model is retrained on the updated data
5. A new recommendation is generated for the user, excluding any restaurants they have already reviewed

This workflow keeps the logic transparent and immediate, while remaining efficient on our small dataset.

The results of this are indeterminate as well, as we do not have a defined random state for the recommender. If we apply a defined random state, the output will be consistent, but we wanted to allow for some randomness so the recommendations are more varied.

1.5 4. Performance

The performance of this recommender is consistent, though not incredibly accurate. The method by which I found the current performance was through Surprises GridSearchCV. This allows us to give a list of parameters, and test for which combination has the best results:

```
from surprise.model_selection import GridSearchCV
from surprise import SVD

param_grid = {
    'n_factors': [120, 140, 160, 180, 200],
    'lr_all': [0.004, 0.006, 0.008, 0.01],
    'reg_all': [0.03, 0.04, 0.05, 0.06],
    'n_epochs': [20, 30, 40]
}

gs = GridSearchCV(SVD, param_grid, measures=['rmse'], cv=5, joblib_verbose=2)
gs.fit(data)

print("Best RMSE:", gs.best_score['rmse'])
print("Best Params:", gs.best_params['rmse'])
```

This allows us to both see what the best parameters are, as well as what the performance is. Performance metrics are as follows:

- The best performance using KNN had an RMSE of 1.09. This tells us that with KNN, the model should be somewhere +/- 1.09 stars of the true rating. This takes into account that predictions that are more incorrect are weighted more heavily, as well.
- SVD had slightly better performance, with or best performance having an RMSE of 0.85. This is not much better, given the range is only 1-5. This does leave a lot of room for improvement, but it does give us a good idea of how they would rate something, making it better than guessing.

This lack of performance is likely due to the sparseness of the data. There are 261 restaurants, and most users only review 1-5 restaurants, with us having a total of 7668 reviews. This leaves an incredibly sparse dataset, meaning finding a model that can accurately fill in the gaps of the reviews is very difficult. The best way of improving this would be to introduce more dense reviews, or to reduce the number of restaurants.

1.6 5. Model Persistence

To keep the application lightweight, the model is not saved between sessions. Since training is fast and the dataset is relatively small, we retrain the model on each update. This eliminates the need for managing model versioning or state between users. Additionally, the model should be retrained after every new review, so saving the model will achieve little to nothing.

1.7 6. Future Improvements

There are multiple areas for this recommender that could be improved:

- Use hybrid recommendation (collaborative + content-based filtering) to improve performance
- Store data in a persistent database (e.g., SQLite or PostgreSQL) rather than a .csv
- Add a login/authentication system for user identity validation, rather than just having a username input
- Incorporate contextual metadata like cuisine type, price range, or location, since the data is not very descriptive as is
- If the dataset scales upwards and there are more users, saving the model may be required, which would be retrained regularly
- Add time weighting so newer reviews are prioritized